

URL SHORTENER WEB APPLICATION

Internship: Data Science with Advanced GenAI Internship

Organization: Innomatics Research Labs

Project Type: Final Project (Basic Version)

Submitted by: Krishna Kishore

Technology Stack: Flask, Python, SQLAlchemy, SQLite, HTML, CSS

Submission Date: 18 January 2026

Contents

1. Introduction	3
2. Problem Statement	3
3. Objectives of the Project	3
3.1 Functional Objectives.....	3
3.2 Non-Functional Objectives	3
4. Tools and Technologies Used.....	4
5. System Architecture	5
6. Database Design	6
6.1 Table: URLMap.....	6
7. Implementation Details.....	6
8. Challenges Faced and Solutions.....	9
8.1 Circular Import Issue	9
8.2 URL Validation	9
9. Conclusion.....	9

1. Introduction

The rapid growth of web applications has increased the need for sharing links efficiently. Many URLs are long, complex, and difficult to remember or share, especially across messaging platforms or social media. This project focuses on developing a simple and efficient URL Shortener Web Application using Flask.

The application allows users to convert long URLs into short, easy-to-share links. It also stores previously shortened URLs, enabling users to view their URL history. The project emphasizes clean backend logic, database persistence, and basic input validation while keeping the system simple and user-friendly.

2. Problem Statement

Long URLs are inconvenient to share and can lead to formatting or usability issues. Manually copying and managing such links is inefficient and error-prone. There is a need for a system that can generate short URLs while maintaining a reliable mapping to the original URLs.

The goal of this project is to design and implement a web-based URL shortening system that:

- Accepts user-provided URLs
- Generates unique short URLs
- Redirects users to the original URL
- Stores URL history for future reference

3. Objectives of the Project

3.1 Functional Objectives

- To shorten long URLs into unique short links
- To validate user input before processing
- To store original and shortened URLs in a database
- To redirect users from short URLs to original URLs

3.2 Non-Functional Objectives

- Maintain simplicity and readability of code
- Ensure data persistence using SQLite
- Avoid unnecessary complexity or overengineering

4. Tools and Technologies Used

The following tools and technologies were used in this project:

- **Python:** Core programming language used for backend logic
- **Flask:** Lightweight web framework used to build the application
- **SQLAlchemy:** ORM used to interact with the SQLite database
- **SQLite:** Lightweight database for storing URLs
- **HTML & CSS:** Used to design the frontend interface

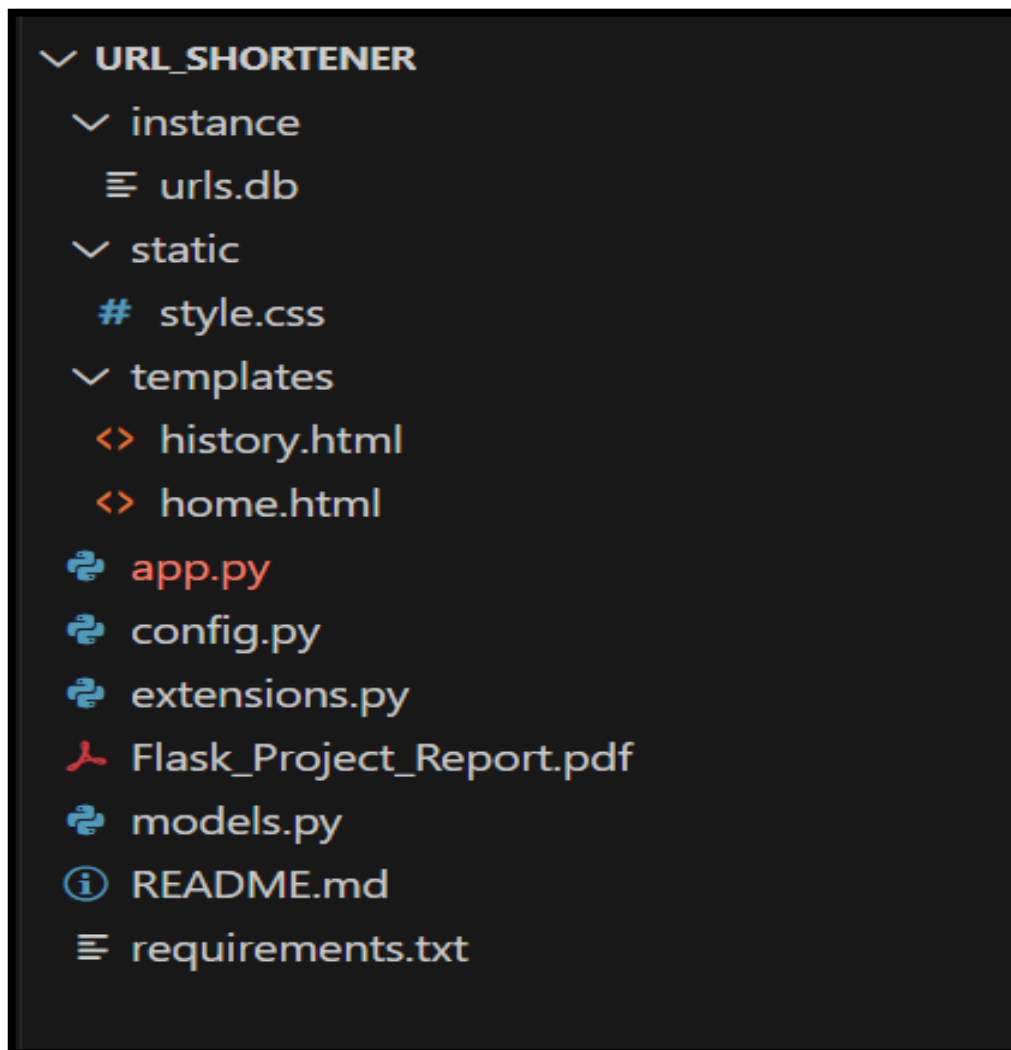


Figure 1: Project folder structure of the URL Shortener application.

5. System Architecture

The application follows a simple client-server architecture. The frontend interacts with the backend through HTTP requests. Flask handles routing, input processing, and responses, while SQLAlchemy manages database interactions.

Workflow Overview:

1. User enters a URL on the homepage
2. The backend validates the URL
3. A unique short code is generated
4. The mapping is stored in the database
5. The short URL is displayed to the user
6. Accessing the short URL redirects to the original URL

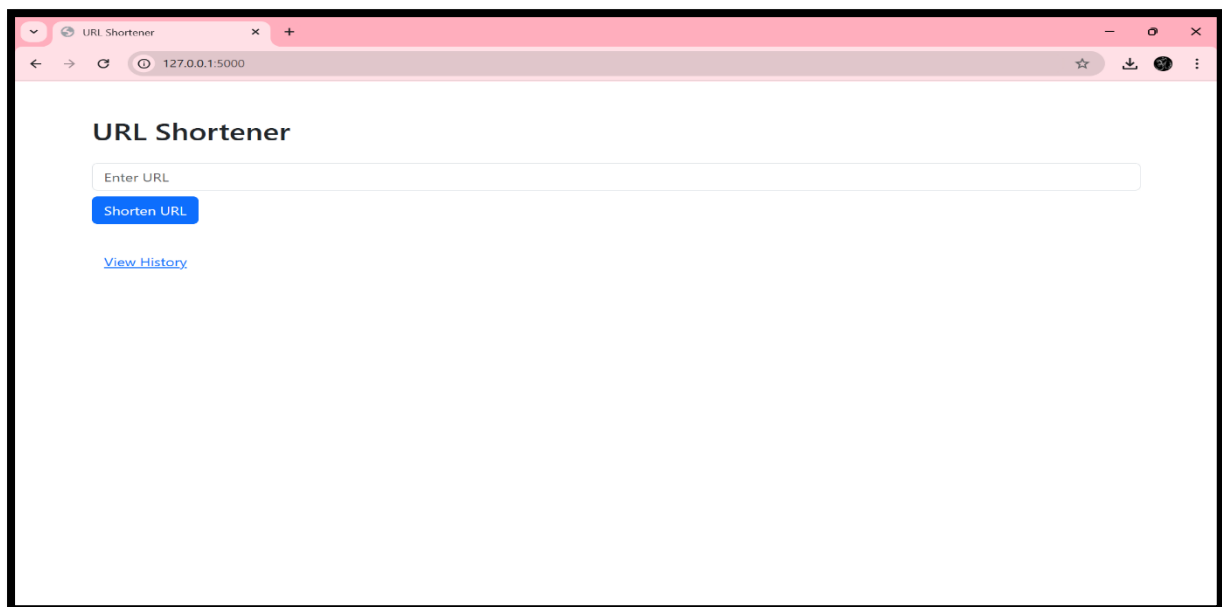


Figure 2: Home page of the URL Shortener web application.

6. Database Design

The application uses a single database table to store URL mappings.

6.1 Table: URLMap

Column Name	Data Type	Description
id	Integer	Primary key
original_url	Text	Original long URL
short_code	String	Unique short identifier
created_at	DateTime	Timestamp of creation

This design keeps the database simple and efficient while supporting the required functionality.

```
models.py > ...
1  from datetime import datetime
2  from extensions import db
3
4  class URLMap(db.Model):
5      __tablename__ = 'url_map'
6
7      id = db.Column(db.Integer, primary_key=True)
8      original_url = db.Column(db.Text, nullable=False)
9      short_code = db.Column(db.String(10), unique=True, nullable=False)
10     created_at = db.Column(db.DateTime, default=datetime.utcnow)
11
12     def __repr__(self):
13         return f"<URLMap {self.short_code}>"
14
```

Figure 3:SQLAlchemy model defining the URL mapping table.

7. Implementation Details

The backend logic is implemented using Flask routes. A random short code is generated using alphanumeric characters to ensure uniqueness. URL validation is performed using Python's `urllib.parse` module to ensure only valid URLs are accepted.

The application also handles edge cases such as invalid input and non-existent short URLs by displaying appropriate messages.

```

19 # ----- Utility Functions -----
20
21 def generate_short_code(length=6):
22     characters = string.ascii_letters + string.digits
23     return ''.join(random.choice(characters) for _ in range(length))
24
25
26 def is_valid_url(url):
27     parsed = urlparse(url)
28     return parsed.scheme in ('http', 'https') and parsed.netloc != ''
29
30
31 # ----- Routes -----
32
33 @app.route('/', methods=['GET', 'POST'])
34 def home():
35     if request.method == 'POST':
36         original_url = request.form.get('original_url')
37
38         if not is_valid_url(original_url):
39             return render_template('home.html', error="Please enter a valid URL")
40
41         short_code = generate_short_code()
42         while URLMap.query.filter_by(short_code=short_code).first():
43             short_code = generate_short_code()
44
45         new_url = URLMap(original_url=original_url, short_code=short_code)  No parameter named "original_url"
46         db.session.add(new_url)
47         db.session.commit()
48
49         short_url = request.host_url + short_code
50         return render_template('home.html', short_url=short_url)
51
52     return render_template('home.html')
53
54
55 @app.route('/<short_code>')
56 def redirect_url(short_code):
57     url_entry = URLMap.query.filter_by(short_code=short_code).first()
58
59     if not url_entry:
60         return "URL not found", 404
61
62     return redirect(url_entry.original_url)
63
64

```

Figure 4: Core Flask routes and utility functions implemented in the application.

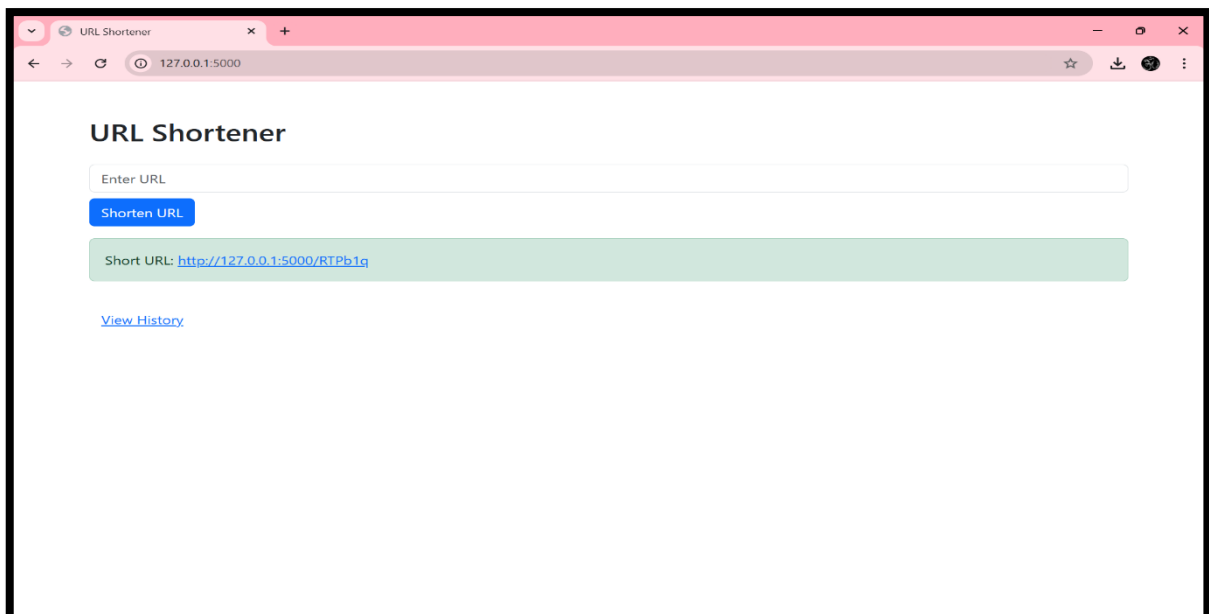


Figure 5: Successful generation of a shortened URL.

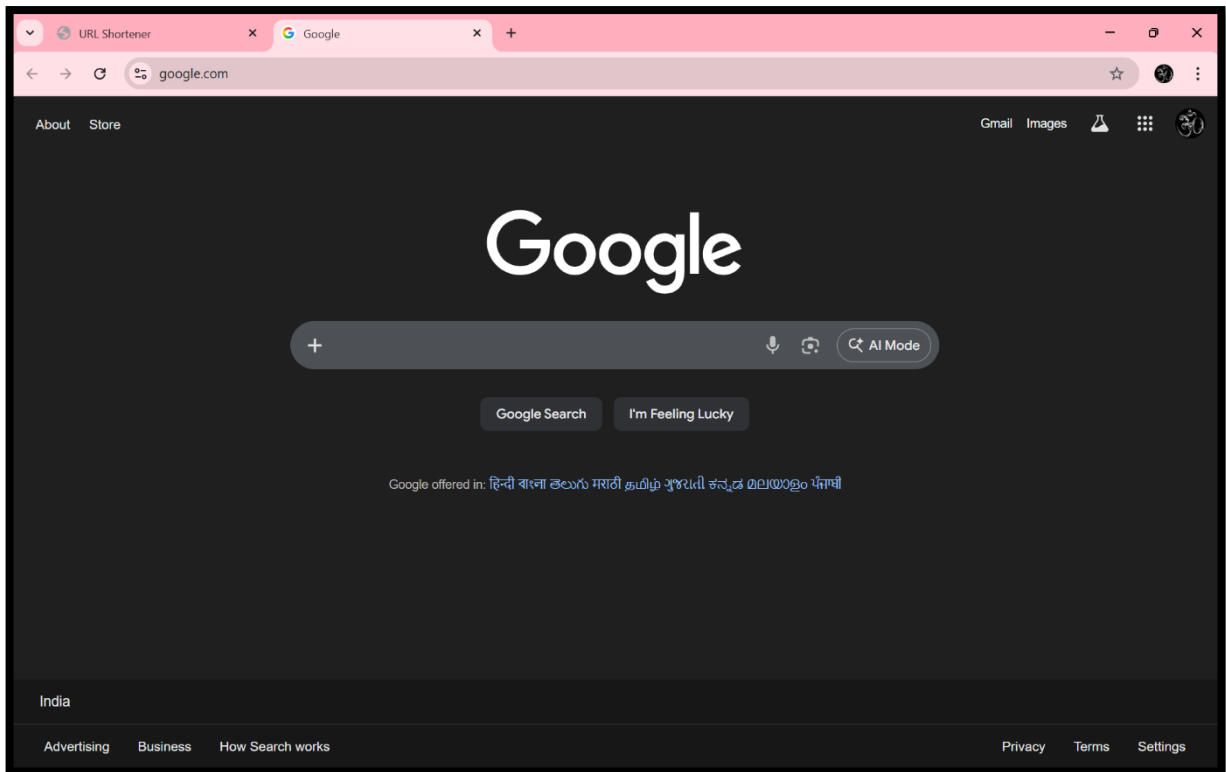


Figure 6: Redirecting from shortened URL to the original website.

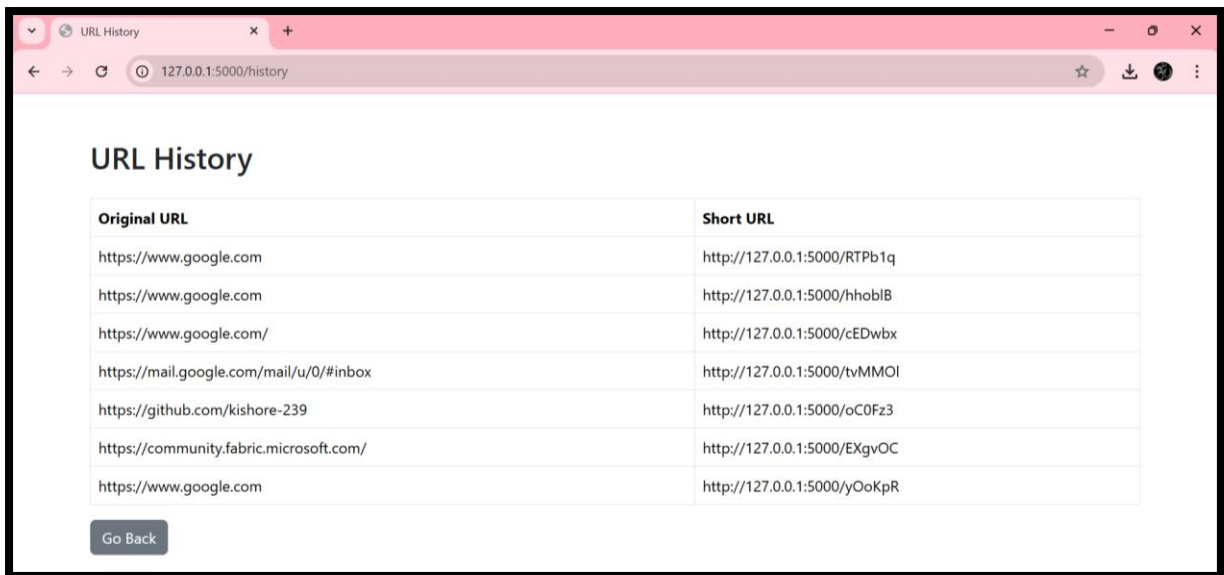


Figure 7: History page displaying previously shortened URLs.

8. Challenges Faced and Solutions

8.1 Circular Import Issue

During development, a circular import issue occurred between the application and model files. This was resolved by separating database initialization into a dedicated module, ensuring clean imports and better structure.

8.2 URL Validation

Ensuring that users enter valid URLs was a challenge. This was handled by checking the scheme and network location of the input URL before processing it.

9. Conclusion

This project successfully demonstrates the development of a basic URL Shortener Web Application using Flask and SQLite. The application meets all functional requirements and maintains a clean and understandable codebase.

Through this project, practical experience was gained in backend development, database handling, and debugging common issues in Flask applications.